

LEARNING RESOURCE GUIDE

Generative AI for Beginners | Professional Participant Workbook

Section	Section 2: Neural Networks - The Foundation
Lecture	Lecture 2.1: How Neural Networks Work - Part 1

Learning Objectives

By the end of this lecture and its accompanying guide, you will be able to:

- Explain the biological inspiration for artificial neural networks and how artificial neurons differ from biological ones.
- Describe the structure of a neural network including input layer, hidden layers, and output layer.
- Articulate how weights and biases function within an individual neuron's computation.
- Explain what an activation function does and why it is necessary.
- Identify how the layered structure of neural networks enables learning increasingly complex patterns.

Introduction

Neural networks are the foundational technology of all modern AI — from the image classifiers in your smartphone to the language models powering ChatGPT and the diffusion models behind DALL-E. Understanding how they work at the level of individual neurons and layers is the prerequisite for understanding every generative AI architecture discussed later in this course. Yet neural networks are frequently treated as impenetrable black boxes even by professionals who use them daily.

The core concept is more accessible than its reputation suggests. A neural network is, at its most fundamental, a mathematical function that takes numbers in and produces numbers out. What makes it remarkable is that the specific form of that function — defined by billions of adjustable parameters called weights and biases — is not programmed by humans. It is learned from data through a process of iterative adjustment. The result is a system that can capture complex, high-dimensional patterns in data that no human programmer could have manually encoded.

Part 1 of this two-lecture series on neural networks focuses on the structural elements: what neurons are, how they compute, how they are organised into layers, and why this layered structure is so powerful. Part 2 will cover how information flows through the network during inference (forward propagation), and Part 3 will cover how the network learns (backpropagation and training).

Core Concepts Explained

The Artificial Neuron

Definition: An artificial neuron is a mathematical unit that receives multiple numerical inputs, multiplies each by a weight, sums the weighted inputs, adds a bias term, and passes the result through an activation function to produce a single output.

Business relevance: Every inference that a neural network makes — from classifying an image to predicting the next word — is ultimately the result of millions or billions of these simple neuron computations running in parallel. Understanding the neuron demystifies what AI is actually computing.

Practitioner relevance: For professionals overseeing AI systems, understanding that each neuron is a weighted sum followed by a nonlinearity is the basis for understanding why neural networks can approximate complex functions and why they require so much data and compute to train effectively.

Real-world example: Consider a neuron deciding whether to flag an email as spam. Its inputs might include word frequency scores, sender reputation, link count, and time of day. Each input is multiplied by its learned weight (reflecting how predictive that feature is of spam), the weighted sum is computed, a bias is added, and an activation function produces a value between 0 and 1 representing the probability of spam.

Risks or limitations: A single neuron can only learn linear relationships. The non-linearity of the activation function — applied after the weighted sum — is what allows neural networks to learn complex, non-linear patterns. Without activation functions, a deep network collapses into a single linear transformation.

Weights and Biases

Definition: Weights are the numerical parameters associated with each connection between neurons, controlling how much influence each input has on the receiving neuron's output. Biases are additional parameters added to each neuron that shift its activation threshold independently of inputs.

Business relevance: Weights and biases are what neural networks learn during training. When a model is said to have 175 billion parameters, it means it has 175 billion weights and biases. These parameters collectively encode the knowledge the model has learned from training data.

Practitioner relevance: The total number of parameters in a model directly affects its capacity (ability to learn complex patterns), its training cost (compute and data requirements), its memory footprint, and its inference speed. These are all factors in evaluating AI deployment feasibility.

Real-world example: In a language model predicting the next word after The cat sat on the, the weight associated with the connection between the word cat and the concept of animate objects will be high, influencing the model to predict words like mat or floor rather than galaxy. These weights were learned by exposure to billions of text examples.

Activation Functions

Definition: An activation function is applied to a neuron's weighted sum to introduce non-linearity into the network. Common activation functions include ReLU (Rectified Linear Unit), sigmoid, and tanh. Without non-linearity, multiple layers would collapse into a single linear transformation.

Business relevance: The choice of activation function affects how quickly a network trains, whether it suffers from the vanishing gradient problem, and its ultimate performance. ReLU and its variants dominate modern architectures because they are computationally efficient and avoid the vanishing gradient problem that plagued early networks.

Practitioner relevance: When evaluating AI systems, knowing that activation functions introduce non-linearity helps explain why neural networks can model complex real-world relationships — and why depth (more layers with non-linear activations) is a source of representational power.

Real-world example: The ReLU activation function simply outputs zero for any negative input and passes positive values through unchanged. Despite its simplicity, ReLU-based networks consistently outperform networks using more complex activation functions on most tasks, a finding that led to widespread adoption across modern architectures.

Risks or limitations: The dying ReLU problem occurs when neurons receive consistently negative inputs and permanently output zero, effectively becoming inactive. Variants like Leaky ReLU and GELU (used in modern transformers) address this limitation.

Layers and Network Depth

Definition: Neural networks are organised into layers: an input layer that receives raw data, one or more hidden layers that learn intermediate representations, and an output layer that produces the final prediction. Deep networks have many hidden layers, enabling them to learn hierarchical features.

Business relevance: Network depth is a key architectural choice. Deeper networks can learn more abstract and complex features — a face recogniser might learn edges in early layers, then shapes, then facial parts, then identities. But deeper networks are also harder to train and more expensive to run.

Practitioner relevance: The concept of hierarchical feature learning in deep networks explains why these systems are so powerful for complex tasks like image recognition and natural language understanding — tasks where the meaningful patterns exist at multiple levels of abstraction simultaneously.

Real-world example: A deep learning model trained to diagnose diabetic retinopathy from retinal images does not need to be explicitly programmed with medical knowledge. Its early layers learn to detect image features like edges and colour gradients; middle layers learn structures like blood vessels; deep layers learn pathological patterns associated with disease — all from labelled examples.

Practical Applications

Neural Networks in Computer Vision

Convolutional neural networks use the layered neuron structure to process images hierarchically. Applications include quality control in manufacturing (detecting defects), medical imaging (identifying tumours), autonomous vehicle perception (detecting pedestrians and obstacles), and facial recognition in security systems.

Neural Networks in Natural Language Processing

Before transformers, recurrent neural networks processed text sequentially through layered neurons. Now, transformer-based models use the same fundamental neuron building blocks but arranged in a novel attention-based architecture. Both approaches ultimately rely on the basic neuron computations described in this lecture.

Evaluating AI Model Architecture Claims

When a vendor claims their AI system uses a 10-billion-parameter deep neural network, understanding that parameters are weights and biases — and that more parameters mean more capacity but also more compute, data, and memory requirements — allows you to ask more informed questions about deployment feasibility and cost.

Best Practices

- Match network depth and width to problem complexity. Deeper and wider networks have more representational capacity but require more data and compute to train. Oversized networks can overfit to training data; undersized ones lack capacity to capture relevant patterns.
- Understand parameter counts in context. A 175-billion-parameter model is powerful but requires massive compute and memory to run. Smaller, task-specific models fine-tuned from large pre-trained models are often more practical and cost-effective for targeted business applications.

- Recognise that network architecture choices are empirical. There is no formula for the optimal network architecture for a given task. Architecture design involves experimentation, benchmarking, and knowledge of what has worked in similar problems.
- Do not conflate parameter count with performance. A larger model is not always better for a specific task. A smaller model trained on domain-specific data often outperforms a much larger general-purpose model on narrow, well-defined tasks.

Common Mistakes and Misconceptions

Misconception: Thinking that the architecture of a neural network is hand-designed to solve the specific problem.

Correct approach: The architecture (number of layers, neurons per layer, activation functions) is designed by engineers, but the parameters (weights and biases) are learned from data through training. The architecture provides the structure; training fills it with knowledge.

Misconception: Equating a neural network with the human brain.

Correct approach: Artificial neural networks are loosely inspired by neuroscience but are fundamentally mathematical constructs. They share some high-level principles (distributed computation, learning from examples) but differ profoundly in mechanism, scale, and capability.

Misconception: Assuming that more layers always improve performance.

Correct approach: Very deep networks can suffer from training difficulties including vanishing gradients and diminishing returns. Architectural innovations like residual connections (skip connections in ResNets) were specifically designed to make very deep networks trainable.

Reference Table: Neural Network Components — Quick Reference

Use this table as a reference when encountering neural network terminology in product documentation, research papers, or vendor conversations.

Component	What it is	Why it matters
Input layer	First layer receiving raw data (pixels, token IDs, numerical features)	Determines what information the network processes
Hidden layers	Intermediate layers learning progressively abstract features	Source of the network's representational power
Output layer	Final layer producing the prediction (class probabilities, token logits, etc.)	Determines what the network predicts
Weights	Learnable parameters on each neuron connection	Encode the relationships learned from training data
Biases	Learnable offset parameters per neuron	Allow activation thresholds to shift independently of inputs
Activation function	Non-linear function applied after weighted sum (ReLU, GELU, sigmoid)	Enables learning of non-linear patterns
Parameters	All weights and biases combined	Total learnable knowledge capacity of the model
Depth	Number of layers	Determines how abstract the features the network can learn are
Width	Neurons per layer	Determines the richness of representations at each level

How to use this: When reading about a neural network model, locate these components in the description. Ask: how many layers? how many parameters? what activation functions? These determine capability, cost, and appropriate use cases.

Knowledge Check

Test your understanding before moving on. Answers are shown below each question.

1. A neural network for email spam detection receives inputs including word frequency scores, sender reputation, and link count. What are these input values multiplied by before being summed in a neuron?

- A. Activation functions
- B. Weights — learnable parameters that reflect each input's predictive importance
- C. Biases — fixed offset values
- D. Layer indices

Answer: B. Weights — learnable parameters that reflect each input's predictive importance

Why: Each input to a neuron is multiplied by its weight before summation. Weights are learned during training and encode how much influence each input should have on the neuron's output.

2. Why is an activation function necessary in a neural network?

- A. It speeds up data loading from the training dataset.
- B. It introduces non-linearity, enabling the network to learn complex patterns beyond simple linear relationships.
- C. It controls the learning rate during training.
- D. It initialises the weights before training begins.

Answer: B. It introduces non-linearity, enabling the network to learn complex patterns beyond simple linear relationships.

Why: Without activation functions, a deep network of linear computations collapses into a single linear transformation, losing all the representational power gained from depth.

3. A deep learning model for medical image diagnosis learns to detect edges in early layers, blood vessel structures in middle layers, and disease-specific patterns in deep layers. Which property of neural networks does this demonstrate?

- A. Overfitting to training data
- B. Hierarchical feature learning through layered non-linear transformations
- C. The vanishing gradient problem
- D. Weight initialisation strategies

Answer: B. Hierarchical feature learning through layered non-linear transformations

Why: Deep networks learn progressively more abstract and complex features in successive layers — edges, then shapes, then semantic concepts — a property called hierarchical feature learning.

Reflection Questions

1. The concept of weights as learned importance scores for each input has analogies in human decision-making. Can you identify a professional decision you make regularly where different inputs have different weights of importance?
2. Neural networks are described as learning from data rather than from explicit rules. What types of problems in your domain are too complex to express as explicit rules but could potentially be learned from examples?
3. The trade-off between network size (parameters) and practical deployment constraints (compute, memory, cost) is a recurring theme. How does this trade-off manifest in technology decisions in your organisation?
4. Activation functions introduce the non-linearity that makes deep learning powerful. What is the professional analogy — where adding a non-linear step in a process dramatically increases its power or flexibility?

Key Takeaways

- An artificial neuron computes a weighted sum of its inputs, adds a bias, and applies an activation function — a simple operation that becomes powerful when millions are connected in layers.
- Weights and biases are the learnable parameters of a neural network — they encode what the model has learned from training data.
- Activation functions introduce non-linearity, enabling neural networks to learn complex, non-linear patterns that linear functions cannot capture.
- Deeper networks learn hierarchical features — from low-level patterns in early layers to high-level abstractions in later layers.
- Parameter count (weights + biases) reflects a model's capacity but also its compute, memory, and data requirements.
- Neural networks do not require domain knowledge to be programmed explicitly — they learn the relevant features from labelled training examples.
- All generative AI systems — LLMs, image generators, audio models — are built on the same fundamental neuron and layer concepts introduced in this lecture.

Recommended Next Actions

5. Draw a diagram of a simple three-layer neural network (input, one hidden layer, output) and label: input layer, hidden layer, output layer, weights on connections, and bias on each neuron.
6. Look up the parameter count for one AI model you use regularly (e.g., GPT-3 has 175 billion). Use the table from this guide to contextualise what those numbers mean.
7. Add to your glossary: Neuron, Weight, Bias, Activation Function, ReLU, Hidden Layer, Parameters, Depth.
8. Reflect on one AI application you have encountered and ask: what are its inputs, what are its outputs, and how many layers of abstraction might be involved in mapping between them?